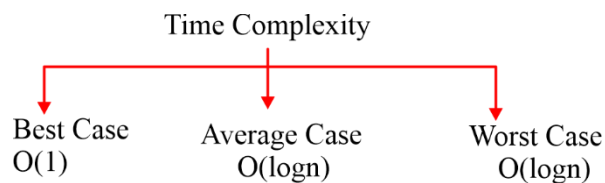
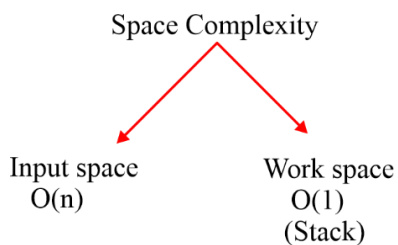




Time Complexity:



Space Complexity:



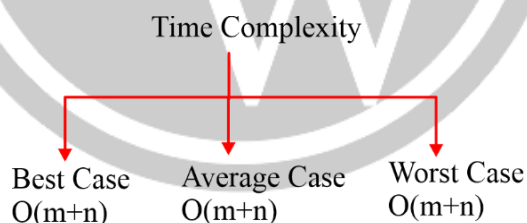
$$\begin{aligned}\text{Space complexity} &= O(n) + O(1) \\ &= O(n)\end{aligned}$$

2.5 Merge Algorithm

- Merging two sorted sub arrays of input size m, n .
- Number of comparisons to merge two sorted sub arrays of size m, n .

$$\text{Comparisons} = m + n - 1 \text{ (worst case)}$$

$$\text{Number of moves} = m + n \text{ (Outplace Algorithm)}$$



Number of comparisons in best case of merging two sorted subarrays of size m, n .

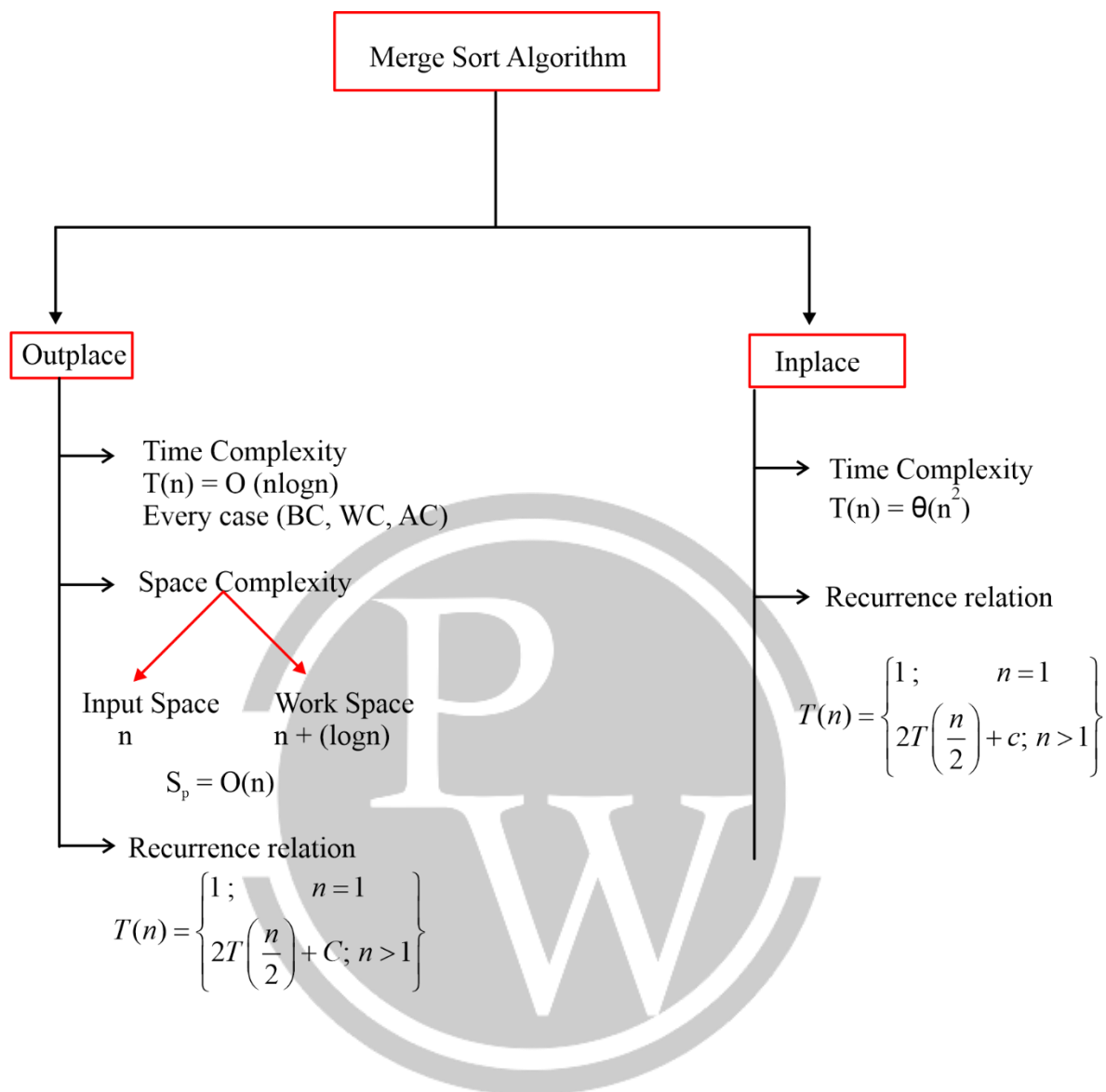
$$\text{comparisons} = \min(m, n)$$

$$\text{Moves} = m + n \text{ (Always)}$$

Note:

Best Case comes in comparisons no effect on moves.

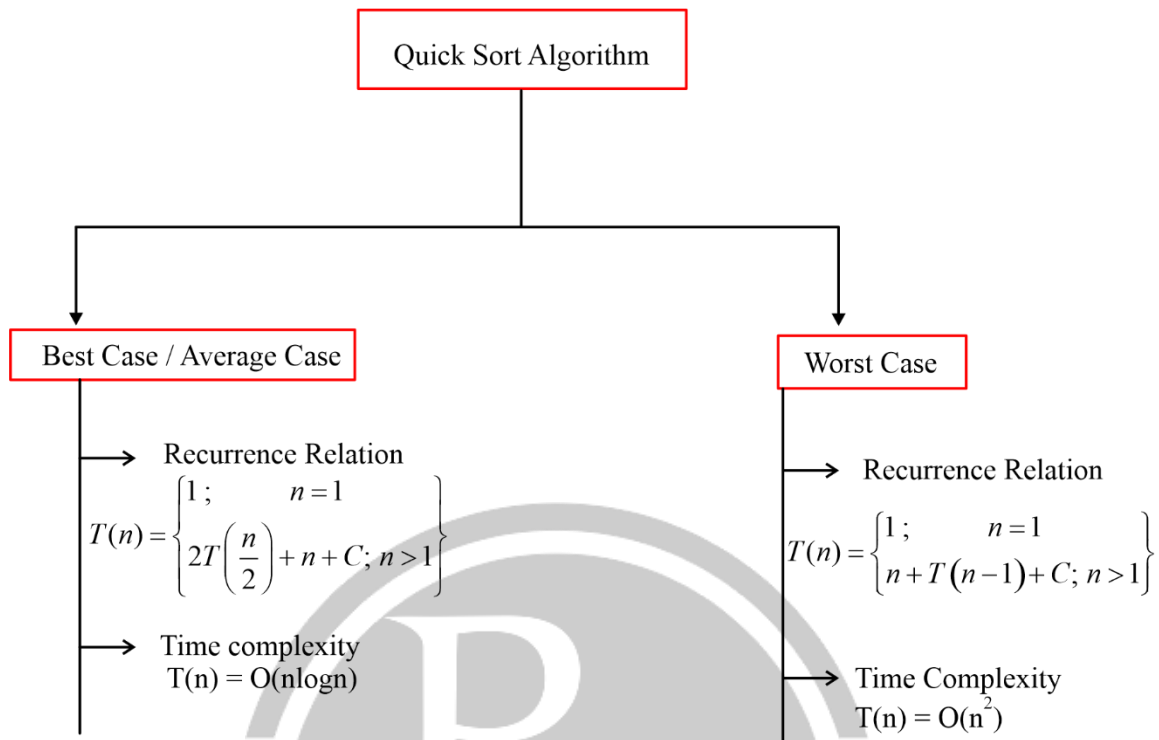
2.5.1 Merge Sort Algorithm:



Note:

- In GATE exam if merge sort given then always consider outplace.
- If array size very large, merge sort preferable.
- If array size very small, then prefer insertion sort.
- Merge sort is stable sorting technique.

2.6 Quick Sort Algorithm



Example 1: In Quick for sorting n elements, the $\left(\frac{n}{16}\right)^{\text{th}}$ smallest element is selected as pivot. what is the worst-case time Complexity?

Solution.

$$T(n) = T\left(\frac{n}{16}\right) + T\left(\frac{15n}{16}\right) + O(n)$$

= (solve by recursive tree method)

Example 2: The median of n elements can be found in $O(n)$ time then, what is the time complexity of quick sort algo in which median selected as pivot?

Solution.

$$T(n) = O(n) + C + O(n) + T(n/2) + T(n/2)$$

↓

Find median

↓

swap median
with last

↓

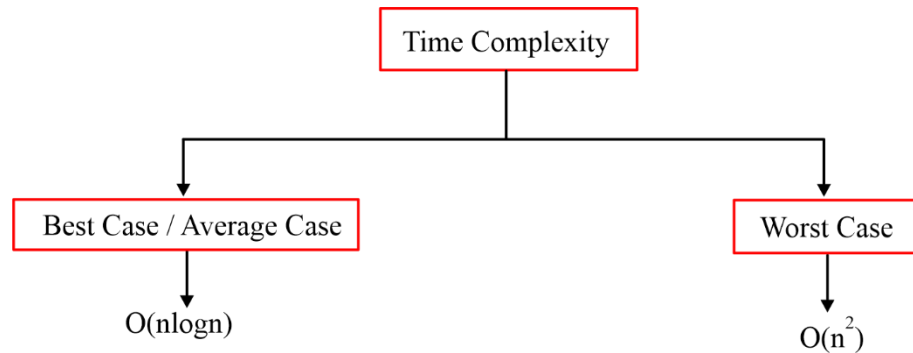
Partition algo

$$= 2T(n/2) + C \cdot n$$

$$= O(n \log n)$$

2.6.1 Randomized Quick Sort

- In Randomized quick sort algorithm selection of pivot element can be taken randomly.



2.7 Counting Number of Inversion

- Counting number of inversion on given an array of an element.

Time complexity $T(n) = O(n \log n)$

2.8 Selection Procedure

Find K^{th} smallest on given an array of an element and integer K .

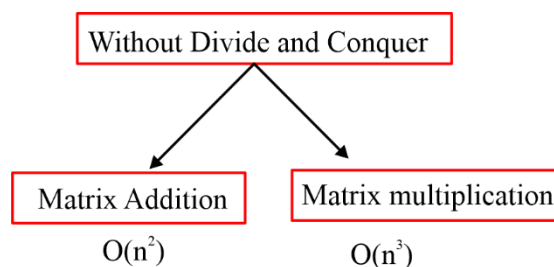
Time Complexity:

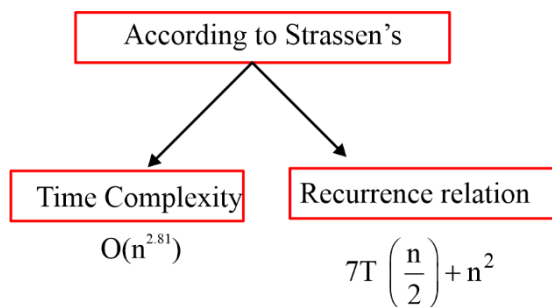
$$T(n) = O(n^2)$$

Space complexity:

$$\text{Space Complexity} = O(n)$$

2.9 Strassen's matrix Multiplication





2.10 Comparison Based Sorting Algorithms

Sorting Algorithm	Basic logic of sorting Algo	BC	AC	WC	Stable sorting	Inplace sorting
Quick sort	Choose pivot element place in correct position	$\theta(n \log n)$	$\theta(n \log n)$	$\theta(n^2)$	No	Yes
Merge sort	Divide to equal parts recursively sort each sub part & marge them	$\theta(n \log n)$	$\theta(n \log n) = n \log n$	$\theta(n \log n) = n \log n$	Yes	No
Heap sort	Build heap(max) delete max place	$\theta(n \log n)$	$\theta(n \log n)$	$\theta(n \log n)$	No	Yes
Bubble sort	Compare exchange	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	Yes	Yes
Selection sort	Find position of min element from [1 to n]	$\theta(n^2)$	$\theta(n^2)$	$\theta(n^2)$	No	Yes
Insertion sort	Insert a [i + 1] into correct position	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	Yes	Yes

□□□